```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import gradio as gr
from datetime import datetime
import matplotlib

# Set matplotlib backend to Agg to prevent issues with GUI threads in web apps
matplotlib.use('Agg')

# ==========================================
# 1. OOP Core and Data Management
# ==========================================

class DealershipManager:
    """
    Core OOP Class handling data operations, logic, and CSV file persistence
    for the Dealership Management System.
    """
    def __init__(self, data_dir="data"):
        self.data_dir = data_dir
        if not os.path.exists(self.data_dir):
            os.makedirs(self.data_dir)
        self.initialize_csv_files()

    def get_file_path(self, filename):
        return os.path.join(self.data_dir, filename)

    def initialize_csv_files(self):
        """Automatically add sample data on first run."""
        # 1. Customers
        if not os.path.exists(self.get_file_path("customers.csv")):
            df = pd.DataFrame(columns=["CustomerID", "Name", "Phone", "Email", "Address"])
            df.loc[0] = ["C001", "John Doe", "9876543210", "john@example.com", "123 Main St, City"]
            df.to_csv(self.get_file_path("customers.csv"), index=False)

        # 2. Bikes
        if not os.path.exists(self.get_file_path("bikes.csv")):
            df = pd.DataFrame(columns=["BikeID", "Model", "EngineCC", "Color", "Price", "Quantity", "Features"])
            sample_bikes = [
                ["B001", "Classic 350", "349cc", "Halcyon Black", 193000, 10, "Dual Channel ABS, Alloy Wheels"],
                ["B002", "Hunter 350", "349cc", "Rebel Blue", 170000, 15, "Tripper Navigation, Alloy Wheels"],
                ["B003", "Bullet 350", "349cc", "Standard Black", 175000, 5, "Single Channel ABS, Spoke Wheels"],
                ["B004", "Meteor 350", "349cc", "Fireball Red", 205000, 8, "Tripper Navigation, Windshield"],
                ["B005", "Interceptor 650", "648cc", "Canyon Red", 303000, 3, "Dual Channel ABS, Slipper Clutch"],
                ["B006", "Continental GT 650", "648cc", "Mr Clean", 319000, 2, "Clip-on Handlebars, Cafe Racer Design"
            ]
            for i, bike in enumerate(sample_bikes):
                df.loc[i] = bike
            df.to_csv(self.get_file_path("bikes.csv"), index=False)

        # 3. Test Rides
        if not os.path.exists(self.get_file_path("test_rides.csv")):
            df = pd.DataFrame(columns=["BookingID", "CustomerID", "Model", "Date", "Time", "Contact"])
            df.loc[0] = ["TR001", "C001", "Classic 350", "2023-10-25", "10:00 AM", "9876543210"]
            df.to_csv(self.get_file_path("test_rides.csv"), index=False)

        # 4. Purchases
        if not os.path.exists(self.get_file_path("purchases.csv")):
            df = pd.DataFrame(columns=["InvoiceNo", "CustomerID", "Model", "Color", "Quantity", "BikePrice", "Accesso
            df.loc[0] = ["INV001", "C001", "Classic 350", "Halcyon Black", 1, 193000, 5000, 12000, 15000, 225000, "UPI
            df.to_csv(self.get_file_path("purchases.csv"), index=False)

        # 5. Payments
```

```
    if not os.path.exists(self.get_file_path("payments.csv")):
        df = pd.DataFrame(columns=["PaymentID", "InvoiceNo", "CustomerID", "Amount", "Method", "Status", "Date"])
        df.loc[0] = ["PAY001", "INV001", "C001", 225000, "UPI", "Completed", "2023-10-26"]
        df.to_csv(self.get_file_path("payments.csv"), index=False)

    # 6. Deliveries
    if not os.path.exists(self.get_file_path("deliveries.csv")):
        df = pd.DataFrame(columns=["DeliveryID", "InvoiceNo", "Model", "DeliveryDate", "Status"])
        df.loc[0] = ["DEL001", "INV001", "Classic 350", "2023-10-28", "Delivered"]
        df.to_csv(self.get_file_path("deliveries.csv"), index=False)

    # 7. Feedback
    if not os.path.exists(self.get_file_path("feedback.csv")):
        df = pd.DataFrame(columns=["FeedbackID", "CustomerID", "InvoiceNo", "Model", "TestRideRating", "SalesRatin
        df.loc[0] = ["FB001", "C001", "INV001", "Classic 350", 5, 4, 5, 5, "Great experience!"]
        df.to_csv(self.get_file_path("feedback.csv"), index=False)

def load_data(self, filename):
    try:
        return pd.read_csv(self.get_file_path(filename))
    except FileNotFoundError:
        return pd.DataFrame()

def save_data(self, df, filename):
    df.to_csv(self.get_file_path(filename), index=False)

# --- Methods for the UI ---

def add_customer(self, name, phone, email, address):
    if not name or not phone: return "Name and Phone are required."
    df = self.load_data("customers.csv")
    new_id = f"C{len(df) + 1:03d}"
    new_row = {"CustomerID": new_id, "Name": name, "Phone": phone, "Email": email, "Address": address}
    df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)
    self.save_data(df, "customers.csv")
    return f"? Customer {name} added successfully! ID: {new_id}"

def search_customer(self, search_term):
    df = self.load_data("customers.csv")
    if not search_term: return df
    # Search by ID or Name
    return df[df["CustomerID"].str.contains(search_term, case=False, na=False) |
              df["Name"].str.contains(search_term, case=False, na=False)]

def add_bike(self, model, engine_cc, color, price, quantity, features):
    if not model or not price: return "Model and Price are required."
    df = self.load_data("bikes.csv")
    new_id = f"B{len(df) + 1:03d}"
    new_row = {"BikeID": new_id, "Model": model, "EngineCC": engine_cc, "Color": color,
               "Price": float(price), "Quantity": int(quantity), "Features": features}
    df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)
    self.save_data(df, "bikes.csv")
    return f"? Bike {model} added successfully! ID: {new_id}"

def get_all_bikes(self):
    return self.load_data("bikes.csv")

def get_bike_models(self):
    df = self.load_data("bikes.csv")
    if df.empty: return []
    return df["Model"].unique().tolist()

def book_test_ride(self, cust_id, model, date, time_slot, contact):
    cust_df = self.load_data("customers.csv")
    bike_df = self.load_data("bikes.csv")
    tr_df = self.load_data("test_rides.csv")
```

```
    # Validation Checks
    if cust_id not in cust_df["CustomerID"].values:
        return "? Error: Customer ID does not exist. Please register."
    if model not in bike_df["Model"].values:
        return "? Error: Bike model not found."

    bike_qty = bike_df.loc[bike_df["Model"] == model, "Quantity"].values[0]
    if bike_qty <= 0:
        return "? Error: Bike is currently out of stock."

    conflict = tr_df[(tr_df["Model"] == model) & (tr_df["Date"] == date) & (tr_df["Time"] == time_slot)]
    if not conflict.empty:
        return "? Error: Time slot already booked for this model."

    new_id = f"TR{len(tr_df) + 1:03d}"
    new_row = {"BookingID": new_id, "CustomerID": cust_id, "Model": model, "Date": date, "Time": time_slot, "Conta
    tr_df = pd.concat([tr_df, pd.DataFrame([new_row])], ignore_index=True)
    self.save_data(tr_df, "test_rides.csv")
    return f"? Success: Test ride booked! Booking ID: {new_id}"

def generate_bill(self, cust_id, model, color, qty, accessories, insurance, registration, payment_method):
    cust_df = self.load_data("customers.csv")
    bike_df = self.load_data("bikes.csv")

    # Validations
    if cust_id not in cust_df["CustomerID"].values:
        return "? Error: Customer not found.", ""
    bike_row = bike_df[bike_df["Model"] == model]
    if bike_row.empty:
        return "? Error: Bike not found.", ""

    bike_price = float(bike_row["Price"].values[0])
    available_qty = int(bike_row["Quantity"].values[0])
    qty = int(qty)

    if available_qty < qty:
        return f"? Error: Insufficient stock. Only {available_qty} available.", ""

    # Numpy Calculations
    subtotal = np.multiply(bike_price, qty)
    total_amount = np.sum([subtotal, float(accessories), float(insurance), float(registration)])

    purchases_df = self.load_data("purchases.csv")
    invoice_no = f"INV{len(purchases_df) + 1:03d}"
    date = datetime.now().strftime("%Y-%m-%d")

    # Log Purchase
    new_purchase = {"InvoiceNo": invoice_no, "CustomerID": cust_id, "Model": model, "Color": color,
                    "Quantity": qty, "BikePrice": bike_price, "Accessories": float(accessories),
                    "Insurance": float(insurance), "Registration": float(registration),
                    "TotalAmount": total_amount, "PaymentMethod": payment_method, "PurchaseDate": date}
    purchases_df = pd.concat([purchases_df, pd.DataFrame([new_purchase])], ignore_index=True)
    self.save_data(purchases_df, "purchases.csv")

    # Log Payment
    payments_df = self.load_data("payments.csv")
    pay_id = f"PAY{len(payments_df) + 1:03d}"
    new_payment = {"PaymentID": pay_id, "InvoiceNo": invoice_no, "CustomerID": cust_id,
                   "Amount": total_amount, "Method": payment_method, "Status": "Completed", "Date": date}
    payments_df = pd.concat([payments_df, pd.DataFrame([new_payment])], ignore_index=True)
    self.save_data(payments_df, "payments.csv")

    # Create Delivery Log
    deliveries_df = self.load_data("deliveries.csv")
    del_id = f"DEL{len(deliveries_df) + 1:03d}"
```

```
        new_del = {"DeliveryID": del_id, "InvoiceNo": invoice_no, "Model": model, "DeliveryDate": "TBD", "Status": "Pe
        deliveries_df = pd.concat([deliveries_df, pd.DataFrame([new_del])], ignore_index=True)
        self.save_data(deliveries_df, "deliveries.csv")

        # Decrease Inventory Stock
        idx = bike_df.index[bike_df["Model"] == model].tolist()[0]
        bike_df.at[idx, "Quantity"] = max(0, available_qty - qty)
        self.save_data(bike_df, "bikes.csv")

        # Generate Bill String
        cust_name = cust_df[cust_df["CustomerID"] == cust_id]["Name"].values[0]
        bill_text = f"""
========================================
    ROYAL ENFIELD SHOWROOM - TAX INVOICE
========================================
Invoice No : {invoice_no}
Date       : {date}
Customer ID: {cust_id} ({cust_name})
----------------------------------------
Model      : {model} ({color})
Quantity   : {qty}
Base Price : ?{bike_price:,.2f}

Subtotal   : ?{subtotal:,.2f}
Accessories: ?{float(accessories):,.2f}
Insurance  : ?{float(insurance):,.2f}
RTO / Reg  : ?{float(registration):,.2f}
----------------------------------------
TOTAL      : ?{total_amount:,.2f}
Payment    : {payment_method}
========================================
Thank you for choosing Royal Enfield!
"""
        return "? Purchase successful!", bill_text

    def update_delivery(self, invoice_no, delivery_date, status):
        df = self.load_data("deliveries.csv")
        if invoice_no not in df["InvoiceNo"].values:
            return "? Invoice not found."
        idx = df.index[df["InvoiceNo"] == invoice_no].tolist()[0]
        df.at[idx, "DeliveryDate"] = delivery_date
        df.at[idx, "Status"] = status
        self.save_data(df, "deliveries.csv")
        return "? Delivery status updated!"

    def submit_feedback(self, cust_id, invoice_no, model, tr_rating, sales_rating, del_rating, overall_rating, comment
        df = self.load_data("feedback.csv")
        new_id = f"FB{len(df) + 1:03d}"
        new_row = {"FeedbackID": new_id, "CustomerID": cust_id, "InvoiceNo": invoice_no, "Model": model,
                   "TestRideRating": int(tr_rating), "SalesRating": int(sales_rating),
                   "DeliveryRating": int(del_rating), "OverallRating": int(overall_rating), "Comments": comments}
        df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)
        self.save_data(df, "feedback.csv")
        return "? Thank you for your feedback!"

    # --- Visualizations (Matplotlib & Seaborn) ---
    def generate_sales_chart(self):
        df = self.load_data("purchases.csv")
        if df.empty: return None
        fig, ax = plt.subplots(figsize=(8, 4))
        sales = df.groupby("Model")["Quantity"].sum().reset_index()
        sns.barplot(data=sales, x="Model", y="Quantity", ax=ax, palette="viridis")
        ax.set_title("Number of Bikes Sold by Model")
        plt.xticks(rotation=45)
        plt.tight_layout()
        return fig
```

```
    def generate_satisfaction_chart(self):
        df = self.load_data("feedback.csv")
        if df.empty: return None
        fig, ax = plt.subplots(figsize=(8, 4))
        sns.histplot(df["OverallRating"], bins=5, kde=True, ax=ax, color="blue")
        ax.set_title("Customer Satisfaction Distribution")
        ax.set_xlabel("Rating (1-5)")
        plt.tight_layout()
        return fig

    def generate_revenue_chart(self):
        df = self.load_data("purchases.csv")
        if df.empty: return None
        fig, ax = plt.subplots(figsize=(8, 4))
        revenue = df.groupby("Model")["TotalAmount"].sum().reset_index()
        plt.pie(revenue["TotalAmount"], labels=revenue["Model"], autopct='%1.1f%%', startangle=140)
        ax.set_title("Total Sales Revenue by Model")
        plt.tight_layout()
        return fig

# Initialize orchestrator
dealer = DealershipManager()

# ==========================================
# 2. Gradio Interface (The 10 Tabs)
# ==========================================

with gr.Blocks() as demo:
    gr.Markdown("# ?? Royal Enfield Dealership Management System")
    gr.Markdown("Welcome! Manage your customers, inventory, sales, and analytics from this unified dashboard.")

    with gr.Tabs():
        # --- Tab 1: Dashboard ---
        with gr.Tab("1. Dashboard"):
            gr.Markdown("### ? Quick Overview")
            with gr.Row():
                tot_cust = gr.Number(label="Total Registered Customers", interactive=False)
                tot_bikes = gr.Number(label="Total Bikes in Stock", interactive=False)
                tot_revenue = gr.Number(label="Total Revenue (INR)", interactive=False)

            def load_dashboard_stats():
                c_df = dealer.load_data("customers.csv")
                b_df = dealer.load_data("bikes.csv")
                p_df = dealer.load_data("purchases.csv")
                return len(c_df), b_df["Quantity"].sum() if not b_df.empty else 0, p_df["TotalAmount"].sum() if not p_

            dash_btn = gr.Button("Refresh Dashboard")
            dash_btn.click(fn=load_dashboard_stats, outputs=[tot_cust, tot_bikes, tot_revenue])
            demo.load(fn=load_dashboard_stats, outputs=[tot_cust, tot_bikes, tot_revenue])

        # --- Tab 2: Customer Management ---
        with gr.Tab("2. Customer Management"):
            with gr.Row():
                with gr.Column():
                    gr.Markdown("### Register New Customer")
                    c_name = gr.Textbox(label="Full Name")
                    c_phone = gr.Textbox(label="Phone Number")
                    c_email = gr.Textbox(label="Email")
                    c_address = gr.Textbox(label="Address")
                    c_add_btn = gr.Button("Register Customer", variant="primary")
                    c_add_res = gr.Textbox(label="Status")
                with gr.Column():
                    gr.Markdown("### Search & View Customers")
                    c_search = gr.Textbox(label="Search by ID or Name")
                    c_search_btn = gr.Button("Search")
```

```
            c_table = gr.Dataframe(value=dealer.load_data("customers.csv"))

    c_add_btn.click(fn=dealer.add_customer, inputs=[c_name, c_phone, c_email, c_address], outputs=c_add_res)
    c_search_btn.click(fn=dealer.search_customer, inputs=c_search, outputs=c_table)

# --- Tab 3: Bike Management ---
with gr.Tab("3. Bike Management"):
    with gr.Row():
        with gr.Column():
            gr.Markdown("### Add New Bike Model")
            b_model = gr.Textbox(label="Model Name")
            b_cc = gr.Textbox(label="Engine CC (e.g., 349cc)")
            b_color = gr.Textbox(label="Color")
            b_price = gr.Number(label="Price (INR)")
            b_qty = gr.Number(label="Quantity")
            b_feat = gr.Textbox(label="Features")
            b_add_btn = gr.Button("Add Bike", variant="primary")
            b_add_res = gr.Textbox(label="Status")
        with gr.Column():
            gr.Markdown("### Inventory")
            b_refresh_btn = gr.Button("Refresh Inventory")
            b_table = gr.Dataframe(value=dealer.load_data("bikes.csv"))

    b_add_btn.click(fn=dealer.add_bike, inputs=[b_model, b_cc, b_color, b_price, b_qty, b_feat], outputs=b_add
    b_refresh_btn.click(fn=dealer.get_all_bikes, outputs=b_table)

# --- Tab 4: Test Ride Booking ---
with gr.Tab("4. Test Ride Booking"):
    with gr.Row():
        with gr.Column():
            gr.Markdown("### Book a Slot")
            tr_cid = gr.Textbox(label="Customer ID")

            # Create dropdown inside a function to allow dynamic updates
            def get_models(): return dealer.get_bike_models()

            tr_model = gr.Dropdown(choices=get_models(), label="Bike Model")
            tr_date = gr.Textbox(label="Date (YYYY-MM-DD)")
            tr_time = gr.Dropdown(choices=["10:00 AM", "11:00 AM", "12:00 PM", "02:00 PM", "04:00 PM"], label=
            tr_contact = gr.Textbox(label="Contact Number")
            tr_btn = gr.Button("Book Test Ride", variant="primary")
            tr_res = gr.Textbox(label="Status")
        with gr.Column():
            gr.Markdown("### Scheduled Test Rides")
            tr_refresh = gr.Button("Refresh")
            tr_table = gr.Dataframe(value=dealer.load_data("test_rides.csv"))

    # Allow dropdown to refresh when opening tab
    demo.load(fn=lambda: gr.Dropdown(choices=dealer.get_bike_models()), outputs=tr_model)
    tr_btn.click(fn=dealer.book_test_ride, inputs=[tr_cid, tr_model, tr_date, tr_time, tr_contact], outputs=tr
    tr_refresh.click(fn=lambda: dealer.load_data("test_rides.csv"), outputs=tr_table)

# --- Tab 5 & 6: Purchase and Billing ---
with gr.Tab("5 & 6. Bike Purchase & Billing"):
    with gr.Row():
        with gr.Column():
            gr.Markdown("### Purchase Details")
            p_cid = gr.Textbox(label="Customer ID")
            p_model = gr.Dropdown(choices=get_models(), label="Bike Model")
            p_color = gr.Textbox(label="Color Selected")
            p_qty = gr.Number(label="Quantity", value=1)
            p_acc = gr.Number(label="Accessories Cost (INR)", value=0)
            p_ins = gr.Number(label="Insurance Cost (INR)", value=0)
            p_reg = gr.Number(label="Registration Charges (INR)", value=0)
            p_pay = gr.Dropdown(choices=["Cash", "UPI", "Debit Card", "Credit Card", "Bank Transfer", "Loan /
            p_btn = gr.Button("Confirm Purchase & Generate Bill", variant="primary")
```

```
            p_res = gr.Textbox(label="Status")
        with gr.Column():
            gr.Markdown("### Generated Bill")
            p_bill = gr.Textbox(label="Invoice", lines=15)

    demo.load(fn=lambda: gr.Dropdown(choices=dealer.get_bike_models()), outputs=p_model)
    p_btn.click(fn=dealer.generate_bill, inputs=[p_cid, p_model, p_color, p_qty, p_acc, p_ins, p_reg, p_pay],

# --- Tab 7: Payment Management ---
with gr.Tab("7. Payment Management"):
    gr.Markdown("### View All Transactions")
    pay_refresh = gr.Button("Refresh Payments")
    pay_table = gr.Dataframe(value=dealer.load_data("payments.csv"))
    pay_refresh.click(fn=lambda: dealer.load_data("payments.csv"), outputs=pay_table)

# --- Tab 8: Delivery Management ---
with gr.Tab("8. Delivery Management"):
    with gr.Row():
        with gr.Column():
            gr.Markdown("### Update Delivery Status")
            d_inv = gr.Textbox(label="Invoice Number")
            d_date = gr.Textbox(label="Delivery Date (YYYY-MM-DD)")
            d_status = gr.Dropdown(choices=["Pending", "Ready for Delivery", "Delivered"], label="Status")
            d_btn = gr.Button("Update Delivery", variant="primary")
            d_res = gr.Textbox(label="System Output")
        with gr.Column():
            gr.Markdown("### Delivery Log")
            d_refresh = gr.Button("Refresh Log")
            d_table = gr.Dataframe(value=dealer.load_data("deliveries.csv"))

    d_btn.click(fn=dealer.update_delivery, inputs=[d_inv, d_date, d_status], outputs=d_res)
    d_refresh.click(fn=lambda: dealer.load_data("deliveries.csv"), outputs=d_table)

# --- Tab 9: Customer Feedback ---
with gr.Tab("9. Customer Feedback"):
    with gr.Row():
        with gr.Column():
            gr.Markdown("### Submit Feedback")
            fb_cid = gr.Textbox(label="Customer ID")
            fb_inv = gr.Textbox(label="Invoice Number")
            fb_model = gr.Dropdown(choices=get_models(), label="Bike Model")
            fb_tr = gr.Slider(1, 5, step=1, label="Test Ride Experience (1-5)")
            fb_sales = gr.Slider(1, 5, step=1, label="Sales Service Rating (1-5)")
            fb_del = gr.Slider(1, 5, step=1, label="Delivery Experience (1-5)")
            fb_overall = gr.Slider(1, 5, step=1, label="Overall Satisfaction (1-5)")
            fb_comments = gr.Textbox(label="Comments")
            fb_btn = gr.Button("Submit Feedback", variant="primary")
            fb_res = gr.Textbox(label="Status")
        with gr.Column():
            gr.Markdown("### Feedback Records")
            fb_refresh = gr.Button("Refresh Feedback")
            fb_table = gr.Dataframe(value=dealer.load_data("feedback.csv"))

    demo.load(fn=lambda: gr.Dropdown(choices=dealer.get_bike_models()), outputs=fb_model)
    fb_btn.click(fn=dealer.submit_feedback, inputs=[fb_cid, fb_inv, fb_model, fb_tr, fb_sales, fb_del, fb_over
    fb_refresh.click(fn=lambda: dealer.load_data("feedback.csv"), outputs=fb_table)

# --- Tab 10: Data Analysis Dashboard ---
with gr.Tab("10. Data Analysis & Insights"):
    gr.Markdown("### ? Visualizing Sales and Satisfaction")
    a_refresh = gr.Button("Generate / Refresh Charts")
    with gr.Row():
        plot1 = gr.Plot(label="Sales by Model")
        plot2 = gr.Plot(label="Revenue Distribution")
    with gr.Row():
        plot3 = gr.Plot(label="Satisfaction Distribution")
```

```
            a_refresh.click(fn=dealer.generate_sales_chart, outputs=plot1)
            a_refresh.click(fn=dealer.generate_revenue_chart, outputs=plot2)
            a_refresh.click(fn=dealer.generate_satisfaction_chart, outputs=plot3)
            # Load automatically
            demo.load(fn=dealer.generate_sales_chart, outputs=plot1)
            demo.load(fn=dealer.generate_revenue_chart, outputs=plot2)
            demo.load(fn=dealer.generate_satisfaction_chart, outputs=plot3)

if __name__ == "__main__":
    # Runs on localhost at port 7860
    demo.launch(server_name="127.0.0.1", server_port=7860, theme=gr.themes.Soft())
```